Programming Assignment 2: Algorithmic Warm-up

Revision: November 10, 2021

Introduction

Welcome to your second programming assignment of the Algorithmic Toolbox at Coursera! It consists of eight programming challenges. Three of them require you just to implement carefully the algorithms covered in the lectures. The remaining challenges will require you to first design an algorithm and then to implement it. For all the challenges, we provide starter solutions in `C++`, `Java`, and `Python3`. These solutions implement straightforward algorithms that usually work only for small inputs. To verify this, you may want to submit these solutions to the grader. This will usually give you a "`time limit exceeded`" message for `Python` starter files and either "`time limit exceeded`" or "`wrong answer`" message for `C++` and `Java` solutions (the reason for wrong answer being an integer overflow issue). Your goal is to replace a naive algorithm with an efficient one. In particular, you may want to use the naive implementation for stress testing your efficient implementation.

In this programming assignment, the grader will show you the input data if your solution fails on any of the tests. This is done to help you to get used to the algorithmic problems in general and get some experience debugging your programs while knowing exactly on which tests they fail. However, for all the following programming assignments, the grader will show the input data only in case your solution fails on one of the first few tests.

Learning Outcomes

Upon completing this programming assignment you will be able to:

1. See the huge difference between a slow algorithm and a fast one.
2. Play with examples where knowing something interesting about a problem helps to design an algorithm that is much faster than a naive one.
3. Implement solutions that work much more faster than straightforward solutions for the following programming challenges:
   (a) compute a small Fibonacci number;
   (b) compute the last digit of a large Fibonacci number;
   (c) compute a huge Fibonacci number modulo m;
   (d) compute the last digit of a sum of Fibonacci numbers;
   (e) compute the last digit of a partial sum of Fibonacci numbers;
   (f) compute the greatest common divisor of two integers;
   (g) compute the least common multiple of two integers.
4. Implement the algorithms covered in the lectures, design new algorithms.
5. Practice implementing, testing, and debugging your solution. In particular, you will find out how in practice, when you implement an algorithm, you bump into unexpected questions and problems not covered by the general description of the algorithm. You will also check your understanding of the algorithm itself and most probably see that there are some aspects you did not think of before you had to actually implement it. You will overcome all those complexities, implement the algorithms, test them, debug, and submit to the system.

Passing Criteria: 4 out of 8

Passing this programming assignment requires passing at least 4 out of 8 programming challenges from this assignment. In turn, passing a programming challenge requires implementing a solution that passes all the tests for this problem in the grader and does so under the time and memory limits specified in the problem statement.

Contents

1 Fibonacci Number **3**

2 Last Digit of a Large Fibonacci Number **4**

3 Greatest Common Divisor **6**

4 Least Common Multiple **7**

5 Fibonacci Number Again **8**

6 Last Digit of the Sum of Fibonacci Numbers **9**

7 Last Digit of the Sum of Fibonacci Numbers Again **10**

8 Last Digit of the Sum of Squares of Fibonacci Numbers **11**

9 Appendix **12**

9.1 Compiler Flags 12

9.2 Frequently Asked Questions 13

1 Fibonacci Number

Problem Introduction

Recall the definition of Fibonacci sequence: $F_0 = 0$, $F_1 = 1$, and $F_i = F_{i-1} + F_{i-2}$ for $i \geq 2$. Your goal in this problem is to implement an efficient algorithm for computing Fibonacci numbers. The starter files for this problem contain an implementation of the following naive recursive algorithm for computing Fibonacci numbers in `C++`, `Java`, and `Python3`:

```
FIBONACCI(n):
if n ≤ 1:
  return n
return FIBONACCI(n − 1) + FIBONACCI(n − 2)
```

Try compiling and running a starter solution on your machine. You will see that computing, say, F_{40} already takes noticeable time.
Another way to appreciate the dramatic difference between an exponential time algorithm and a polynomial time algorithm is to use the following visualization by David Galles: http://www.cs.usfca.edu/~galles/visualization/DPFib.html. Try computing F_{20} by a recursive algorithm by entering "20" and pressing the "Fibonacci Recursive" button. You will see an endless number of recursive calls. Now, press "Skip Forward" to stop the current algorithm and call the iterative algorithm by pressing "Fibonacci Table". This will compute F_{20} very quickly. (Note that the visualization uses a slightly different definition of Fibonacci numbers: $F_0 = 1$ instead of $F_0 = 0$. This of course has almost no influence on the running time.)

Problem Description

Task. Given an integer n, find the nth Fibonacci number F_n.

Input Format. The input consists of a single integer n.

Constraints. $0 \leq n \leq 45$.

Output Format. Output F_n.

Sample 1.

Input:

```
10
```

Output:

```
55
```

$F_{10} = 55$.

Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

2 Last Digit of a Large Fibonacci Number

Problem Introduction

Your goal in this problem is to find the last digit of n-th Fibonacci number. Recall that Fibonacci numbers grow exponentially fast. For example,

$$F_{200} = 280\,571\,172\,992\,510\,140\,037\,611\,932\,413\,038\,677\,189\,525\,.$$

Therefore, a solution like

```
F[0] ← 0
F[1] ← 1
for i from 2 to n:
  F[i] ← F[i − 1] + F[i − 2]
print(F[n] mod 10)
```

will turn out to be too slow, because as i grows the ith iteration of the loop computes the sum of longer and longer numbers. Also, for example, F_{1000} does not fit into the standard C++ `int` type. To overcome this difficulty, you may want to store in $F[i]$ not the ith Fibonacci number itself, but just its last digit (that is, F_i mod 10). Computing the last digit of F_i is easy: it is just the last digit of the sum of the last digits of F_{i-1} and F_{i-2}:

```
F[i] ← (F[i − 1] + F[i − 2]) mod 10
```

This way, all $F[i]$'s are just digits, so they fit perfectly into any standard integer type, and computing a sum of $F[i-1]$ and $F[i-2]$ is performed very quickly.

Problem Description

Task. Given an integer n, find the last digit of the nth Fibonacci number F_n (that is, F_n mod 10).

Input Format. The input consists of a single integer n.

Constraints. $0 \le n \le 10^7$.

Output Format. Output the last digit of F_n.

Sample 1.

Input:

```
3
```

Output:

```
2
```

$F_3 = 2$.

Sample 2.

Input:

```
331
```

Output:

```
9
```

$F_{331} = 668\,996\,615\,388\,005\,031\,531\,000\,081\,241\,745\,415\,306\,766\,517\,246\,774\,551\,964\,595\,292\,186\,469.$

Sample 3.

Input:

```
327305
```

Output:

```
5
```

F_{327305} does not fit into one line of this pdf, but its last digit is equal to 5.

Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

3 Greatest Common Divisor

Problem Introduction

The greatest common divisor $\mathrm{GCD}(a, b)$ of two non-negative integers a and b (which are not both equal to 0) is the greatest integer d that divides both a and b. Your goal in this problem is to implement the Euclidean algorithm for computing the greatest common divisor.
Efficient algorithm for computing the greatest common divisor is an important basic primitive of commonly used cryptographic algorithms like RSA.

$$\begin{aligned} &\mathrm{GCD}(1344, 217) \\ =&\mathrm{GCD}(217, 42) \\ =&\mathrm{GCD}(42, 7) \\ =&\mathrm{GCD}(7, 0) \\ =&7 \end{aligned}$$

Problem Description

Task. Given two integers a and b, find their greatest common divisor.

Input Format. The two integers a, b are given in the same line separated by space.

Constraints. $1 \leq a, b \leq 2 \cdot 10^9$.

Output Format. Output $\mathrm{GCD}(a, b)$.

Sample 1.

Input:

```
18 35
```

Output:

```
1
```

18 and 35 do not have common non-trivial divisors.

Sample 2.

Input:

```
28851538 1183019
```

Output:

```
17657
```

$28851538 = 17657 \cdot 1634$, $1183019 = 17657 \cdot 67$.

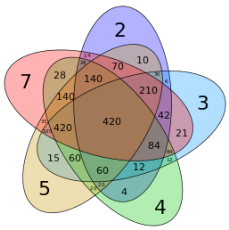
Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

4 Least Common Multiple

Problem Introduction

The least common multiple of two positive integers a and b is the least positive integer m that is divisible by both a and b.



Problem Description

Task. Given two integers a and b, find their least common multiple.

Input Format. The two integers a and b are given in the same line separated by space.

Constraints. $1 \leq a, b \leq 10^7$.

Output Format. Output the least common multiple of a and b.

Sample 1.

Input:

```
6 8
```

Output:

```
24
```

Among all the positive integers that are divisible by both 6 and 8 (e.g., 48, 480, 24), 24 is the smallest one.

Sample 2.

Input:

```
761457 614573
```

Output:

```
467970912861
```

Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

5 Fibonacci Number Again

Problem Introduction

In this problem, your goal is to compute F_n modulo m, where n may be really huge: up to 10^{14}. For such values of n, an algorithm looping for n iterations will not fit into one second for sure. Therefore we need to avoid such a loop.

To get an idea how to solve this problem without going through all F_i for i from 0 to n, let's see what happens when m is small — say, $m = 2$ or $m = 3$.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
F_i	0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610
F_i mod 2	0	1	1	0	1	1	0	1	1	0	1	1	0	1	1	0
F_i mod 3	0	1	1	2	0	2	2	1	0	1	1	2	0	2	2	1

Take a detailed look at this table. Do you see? Both these sequences are periodic! For $m = 2$, the period is 011 and has length 3, while for $m = 3$ the period is 01120221 and has length 8. Therefore, to compute, say, F_{2015} mod 3 we just need to find the remainder of 2015 when divided by 8. Since $2015 = 251 \cdot 8 + 7$, we conclude that $F_{2015} \bmod 3 = F_7 \bmod 3 = 1$.

This is true in general: for any integer $m \geq 2$, the sequence $F_n \bmod m$ is periodic. The period always starts with 01 and is known as Pisano period.

Problem Description

Task. Given two integers n and m, output $F_n \bmod m$ (that is, the remainder of F_n when divided by m).

Input Format. The input consists of two integers n and m given on the same line (separated by a space).

Constraints. $1 \leq n \leq 10^{14}$, $2 \leq m \leq 10^3$.

Output Format. Output $F_n \bmod m$.

Sample 1.

Input:

```
239 1000
```

Output:

```
161
```

$F_{239} \bmod 1\,000 = 39\,679\,027\,332\,006\,820\,581\,608\,740\,953\,902\,289\,877\,834\,488\,152\,161 \pmod{1\,000} = 161.$

Sample 2.

Input:

```
2816213588 239
```

Output:

```
151
```

$F_{2\,816\,213\,588}$ does not fit into one page of this file, but $F_{2\,816\,213\,588} \bmod 239 = 151$.

Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

6 Last Digit of the Sum of Fibonacci Numbers

Problem Introduction

The goal in this problem is to find the last digit of a sum of the first n Fibonacci numbers.

Problem Description

Task. Given an integer n, find the last digit of the sum $F_0 + F_1 + \cdots + F_n$.

Input Format. The input consists of a single integer n.

Constraints. $0 \leq n \leq 10^{14}$.

Output Format. Output the last digit of $F_0 + F_1 + \cdots + F_n$.

Sample 1.

Input:

```
3
```

Output:

```
4
```

$F_0 + F_1 + F_2 + F_3 = 0 + 1 + 1 + 2 = 4$.

Sample 2.

Input:

```
100
```

Output:

```
5
```

The sum is equal to 927 372 692 193 078 999 175, the last digit is 5.

What To Do

Instead of computing this sum in a loop, try to come up with a formula for $F_0 + F_1 + F_2 + \cdots + F_n$. For this, play with small values of n. Then, use a solution for the previous problem.

Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

Solution

A detailed solution (with Python code) for this challenge is covered in the companion MOOCBook. We strongly encourage you to do your best to solve the challenge yourself before looking into the book! There are at least three good reasons for this.

- By solving this challenge, you practice solving algorithmic problems similar to those given at technical interviews.
- The satisfaction and self confidence that you get when passing the grader is priceless =)
- Even if you fail to pass the grader yourself, the time will not be lost as you will better understand the solution from the book and better appreciate the beauty of the underlying ideas.

7 Last Digit of the Sum of Fibonacci Numbers Again

Problem Introduction

Now, we would like to find the last digit of a *partial* sum of Fibonacci numbers: $F_m + F_{m+1} + \cdots + F_n$.

Problem Description

Task. Given two non-negative integers m and n, where $m \leq n$, find the last digit of the sum $F_m + F_{m+1} + \cdots + F_n$.

Input Format. The input consists of two non-negative integers m and n separated by a space.

Constraints. $0 \leq m \leq n \leq 10^{14}$.

Output Format. Output the last digit of $F_m + F_{m+1} + \cdots + F_n$.

Sample 1.

Input:

```
3 7
```

Output:

```
1
```

$F_3 + F_4 + F_5 + F_6 + F_7 = 2 + 3 + 5 + 8 + 13 = 31$.

Sample 2.

Input:

```
10 10
```

Output:

```
5
```

$F_{10} = 55$.

Sample 3.

Input:

```
10 200
```

Output:

```
2
```

$F_{10} + F_{11} + \cdots + F_{200} = 734\,544\,867\,157\,818\,093\,234\,908\,902\,110\,449\,296\,423\,262$

Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

8 Last Digit of the Sum of Squares of Fibonacci Numbers

Problem Description

Task. Compute the last digit of $F_0^2 + F_1^2 + \cdots + F_n^2$.

Input Format. Integer n.

Constraints. $0 \le n \le 10^{14}$.

Output Format. The last digit of $F_0^2 + F_1^2 + \cdots + F_n^2$.

Sample 1.

Input:

```
7
```

Output:

```
3
```

$F_0^2 + F_1^2 + \cdots + F_7^2 = 0 + 1 + 1 + 4 + 9 + 25 + 64 + 169 = 273.$

Sample 2.

Input:

```
73
```

Output:

```
1
```

$F_0^2 + \cdots + F_{73}^2 = 1\,052\,478\,208\,141\,359\,608\,061\,842\,155\,201.$

Sample 3.

Input:

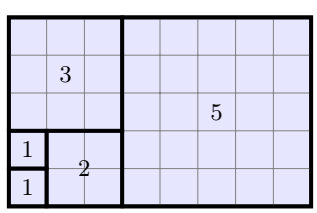
```
1234567890
```

Output:

```
0
```

What To Do

Since the brute force search algorithm for this problem is too slow (n may be as large as 10^{18}), we need to come up with a simple formula for $F_0^2 + F_1^2 + \cdots + F_n^2$. The figure below represents the sum $F_1^2 + F_2^2 + F_3^2 + F_4^2 + F_5^2$ as the area of a rectangle with vertical side $F_5 = 5$ and horizontal side $F_5 + F_4 = 3 + 5 = F_6$.



Need Help?

Ask a question or see the questions asked by other learners at this forum thread.

9 Appendix

9.1 Compiler Flags

C (`gcc 7.4.0`). File extensions: `.c`. Flags:

```
gcc -pipe -O2 -std=c11 <filename> -lm
```

C++ (`g++ 7.4.0`). File extensions: `.cc`, `.cpp`. Flags:

```
g++ -pipe -O2 -std=c++14 <filename> -lm
```

If your `C/C++` compiler does not recognize `-std=c++14` flag, try replacing it with `-std=c++0x` flag or compiling without this flag at all (all starter solutions can be compiled without it). On Linux and MacOS, you most probably have the required compiler. On Windows, you may use your favorite compiler or install, e.g., `cygwin`.

C# (`mono 4.6.2`). File extensions: `.cs`. Flags:

```
mcs
```

Go (`golang 1.13.4`). File extensions: `.go`. Flags

```
go
```

Haskell (`ghc 8.0.2`). File extensions: `.hs`. Flags:

```
ghc -O2
```

Java (`OpenJDK 1.8.0_232`). File extensions: `.java`. Flags:

```
javac -encoding UTF-8
java -Xmx1024m
```

JavaScript (`NodeJS 12.14.0`). File extensions: `.js`. No flags:

```
nodejs
```

Kotlin (`Kotlin 1.3.50`). File extensions: `.kt`. Flags:

```
kotlinc
java -Xmx1024m
```

Python (`CPython 3.6.9`). File extensions: `.py`. No flags:

```
python3
```

Ruby (`Ruby 2.5.1p57`). File extensions: `.rb`.

```
ruby
```

Rust (`Rust 1.37.0`). File extensions: `.rs`.

```
rustc
```

Scala (`Scala 2.12.10`). File extensions: `.scala`.

```
scalac
```

9.2 Frequently Asked Questions

Why My Submission Is Not Graded?

You need to create a submission and upload the *source file* (rather than the executable file) of your solution. Make sure that after uploading the file with your solution you press the blue "Submit" button at the bottom. After that, the grading starts, and the submission being graded is enclosed in an orange rectangle. After the testing is finished, the rectangle disappears, and the results of the testing of all problems are shown.

What Are the Possible Grading Outcomes?

There are only two outcomes: "pass" or "no pass." To pass, your program must return a correct answer on all the test cases we prepared for you, and do so under the time and memory constraints specified in the problem statement. If your solution passes, you get the corresponding feedback "Good job!" and get a point for the problem. Your solution fails if it either crashes, returns an incorrect answer, works for too long, or uses too much memory for some test case. The feedback will contain the index of the first test case on which your solution failed and the total number of test cases in the system. The tests for the problem are numbered from 1 to the total number of test cases for the problem, and the program is always tested on all the tests in the order from the first test to the test with the largest number.

Here are the possible outcomes:

- `Good job!` Hurrah! Your solution passed, and you get a point!
- `Wrong answer.` Your solution outputs incorrect answer for some test case. Check that you consider all the cases correctly, avoid integer overflow, output the required white spaces, output the floating point numbers with the required precision, don't output anything in addition to what you are asked to output in the output specification of the problem statement.
- `Time limit exceeded.` Your solution worked longer than the allowed time limit for some test case. Check again the running time of your implementation. Test your program locally on the test of maximum size specified in the problem statement and check how long it works. Check that your program doesn't wait for some input from the user which makes it to wait forever.
- `Memory limit exceeded.` Your solution used more than the allowed memory limit for some test case. Estimate the amount of memory that your program is going to use in the worst case and check that it does not exceed the memory limit. Check that your data structures fit into the memory limit. Check that you don't create large arrays or lists or vectors consisting of empty arrays or empty strings, since those in some cases still eat up memory. Test your program locally on the tests of maximum size specified in the problem statement and look at its memory consumption in the system.
- `Cannot check answer. Perhaps the output format is wrong.` This happens when you output something different than expected. For example, when you are required to output either "Yes" or "No", but instead output 1 or 0. Or your program has empty output. Or your program outputs not only the correct answer, but also some additional information (please follow the exact output format specified in the problem statement). Maybe your program doesn't output anything, because it crashes.
- `Unknown signal 6 (or 7, or 8, or 11, or some other).` This happens when your program crashes. It can be because of a division by zero, accessing memory outside of the array bounds, using uninitialized variables, overly deep recursion that triggers a stack overflow, sorting with a contradictory comparator, removing elements from an empty data structure, trying to allocate too much memory, and many other reasons. Look at your code and think about all those possibilities. Make sure that you use the same compiler and the same compiler flags as we do.
- `Internal error: exception...` Most probably, you submitted a compiled program instead of a source code.

- `Grading failed.` Something wrong happened with the system. Report this through Coursera or edX Help Center.

May I Post My Solution at the Forum?

Please do not post any solutions at the forum or anywhere on the web, even if a solution does not pass the tests (as in this case you are still revealing parts of a correct solution). Our students follow the Honor Code: "I will not make solutions to homework, quizzes, exams, projects, and other assignments available to anyone else (except to the extent an assignment explicitly permits sharing solutions)."

Do I Learn by Trying to Fix My Solution?

My implementation always fails in the grader, though I already tested and stress tested it a lot. Would not it be better if you gave me a solution to this problem or at least the test cases that you use? I will then be able to fix my code and will learn how to avoid making mistakes. Otherwise, I do not feel that I learn anything from solving this problem. I am just stuck.

First of all, learning from your mistakes is one of the best ways to learn.

The process of trying to invent new test cases that might fail your program is difficult but is often enlightening. Thinking about properties of your program makes you understand what happens inside your program and in the general algorithm you're studying much more.

Also, it is important to be able to find a bug in your implementation without knowing a test case and without having a reference solution, just like in real life. Assume that you designed an application and an annoyed user reports that it crashed. Most probably, the user will not tell you the exact sequence of operations that led to a crash. Moreover, there will be no reference application. Hence, it is important to learn how to find a bug in your implementation yourself, without a magic oracle giving you either a test case that your program fails or a reference solution. We encourage you to use programming assignments in this class as a way of practicing this important skill.

If you have already tested your program on all corner cases you can imagine, constructed a set of manual test cases, applied stress testing, etc, but your program still fails, try to ask for help on the forum. We encourage you to do this by first explaining what kind of corner cases you have already considered (it may happen that by writing such a post you will realize that you missed some corner cases!), and only afterwards asking other learners to give you more ideas for tests cases.